Rate Limiting

Lernunterlage: Anfragen begrenzen mit dem eingebauten Rate Limiter von ASP.NET Core

1 Worum es geht

Rate Limiting begrenzt, wie viele Anfragen in einer bestimmten Zeit erlaubt sind, zum Beispiel „höchstens 10 Anmeldeversuche pro Minute“. Wer mehr schickt, bekommt den Statuscode **429 Too Many Requests**, und der Endpunkt läuft gar nicht erst.

Das Security-Review (REVIEW.md) hatte zwei Lücken gefunden:

- **M-2:** Jeder Aufruf unter `/api/tuya` fragt die Tuya-Cloud. Eine Schleife könnte das Kontingent des Tuya-Projekts aufbrauchen. Danach gehen Status-LEDs und Abgleich nicht mehr.
- **M-3:** Gegen Passwort-Raten schützte nur die Kontosperre (5 Fehlversuche, 5 Minuten). Viele Konten nacheinander durchzuprobieren, war ungebremst möglich.

2 Die Verfahren im Vergleich

Der Rate Limiter von ASP.NET Core (Namespace `System.Threading.RateLimiting`) kennt vier Verfahren:

Verfahren	So funktioniert es	Stärke	Schwäche
Fixed Window (gewählt)	Ein Zähler pro festem Zeitfenster (z. B. 60 s). Am Ende des Fensters zurück auf 0.	Einfach, gut erklärbar, Retry-After eindeutig	Am Fensterrand bis zu doppelt so viele Anfragen möglich
Sliding Window	Das Fenster ist in Segmente geteilt (z. B. 6 × 10 s) und rutscht mit. Alte Segmente fallen weg.	Gleichmäßiger, kein Randeffekt	Etwas schwerer zu erklären
Token Bucket	Ein Eimer mit Marken. Jede Anfrage nimmt eine, in festen Abständen kommen neue dazu.	Erlaubt kurze Spitzen, danach gleichmäßig	Zwei Werte (Größe, Nachfüllrate) einzustellen
Concurrency	Begrenzt, wie viele Anfragen GLEICHZEITIG laufen, nicht pro Zeit.	Schützt langsame Ressourcen	Begrenzt keine Menge pro Minute

Der Randeffekt beim Fixed Window

```
Grenze 10 pro 60 s
Fenster 1: 00:00–01:00           Fenster 2: 01:00–02:00
           ......... 10 × um 00:59 | 10 × um 01:00 .........
→ 20 Anfragen in 2 Sekunden, trotzdem alles erlaubt
```

Für Home Control ist das kein Problem: Auch 20 Login-Versuche in zwei Sekunden sind weit weg von den tausenden, die ein Angreifer bräuchte. Wichtiger war ein Verfahren, das man in der Präsentation in einem Satz erklären kann.

3 429 und Retry-After

Statuscode	Bedeutung	Hier verwendet
401 Unauthorized	Nicht angemeldet (oder Login falsch)	falsches Passwort, kein Token
403 Forbidden	Angemeldet, aber nicht berechtigt	User statt Admin
429 Too Many Requests	Zu viele Anfragen, später wieder versuchen	Rate Limit, auch Kontosperre
503 Service Unavailable	Dienst nicht verfügbar	Standard des Rate Limiters, hier auf 429 geändert

Der Header **Retry-After** sagt dem Client, nach wie vielen Sekunden er es wieder versuchen darf. Der Fixed-Window-Limiter kennt das Ende des aktuellen Fensters und liefert den Wert als Metadaten mit:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 42
Content-Type: application/problem+json

{ "status": 429,
  "title": "Zu viele Anmeldeversuche. Bitte in 42 Sekunden erneut versuchen.",
  "detail": "Diese Grenze schützt vor Passwort-Raten ..." }
```

4 Der Code

RateLimiting/RateLimitingRegistration.cs: die Regeln

```
services.AddRateLimiter(options =>
{
    options.RejectionStatusCode = StatusCodes.Status429TooManyRequests;

    // pro Gerät: Schlüssel = Client-IP
    options.AddPolicy("login", context =>
        RateLimitPartition.GetFixedWindowLimiter(
            context.Connection.RemoteIpAddress?.ToString() ?? "unbekannt",
            _ => CreateFixedWindow(limit)));

    // für alle gemeinsam: immer derselbe Schlüssel
    options.AddPolicy("tuya", context =>
        RateLimitPartition.GetFixedWindowLimiter("alle", _ => CreateFixedWindow(limit)));

    options.OnRejected = WriteRejectionAsync;   // 429 + ProblemDetails + Retry-After
});
```

- **Partition:** Jeder Schlüssel bekommt seinen eigenen Zähler. Gleicher Schlüssel = gleicher Topf.
- **QueueLimit = 0:** Überzählige Anfragen warten nicht in einer Schlange, sie bekommen sofort 429.
- **Grenzen aus appsettings.json** (Abschnitt `RateLimiting`), gelesen erst bei der Anfrage. Werte 0 oder kleiner verhindern den Start (`ValidateOnStart`).

An den Endpunkten

```
auth.MapPost("/login", Login).AllowAnonymous().RequireRateLimiting("login");
auth.MapPost("/register", Register).AllowAnonymous().RequireRateLimiting("login");
auth.MapPost("/refresh", Refresh).AllowAnonymous().RequireRateLimiting("login");

var tuya = app.MapGroup("/api/tuya").RequireRateLimiting("tuya");   // ganze Gruppe
```

login, register und refresh teilen sich **einen** Zähler, weil sie dieselbe Regel und denselben Schlüssel benutzen. GET `/api/status` bekommt kein Limit: Der TuyaOnlineCache fragt Tuya ohnehin höchstens einmal pro Minute, ein Limit würde nur die LEDs mit 429 rot färben.

5 Reihenfolge der Middleware

```
app.UseForwardedHeaders();   // 1. echte Client-IP setzen (siehe Abschnitt 6)
app.UseExceptionHandler();
app.UseStatusCodePages();
app.UseStaticFiles(...);
app.UseAuthentication();      // Wer bist du?
app.UseAuthorization();       // Darfst du das?   → sonst 401/403
app.UseRateLimiter();         // Wie oft schon?   → sonst 429
// ... Endpunkte
```

> Warum der Rate Limiter NACH der Autorisierung steht: POST /api/tuya/sync verlangt eine Anmeldung. Stünde der Limiter davor, könnte ein Fremder ohne Konto den gemeinsamen Tuya-Zähler leer laufen lassen, und alle angemeldeten Benutzer bekämen 429. So bekommt er schon vorher 401 und verbraucht nichts. Der Test Tuya_Anfragen_ohne_Anmeldung_verbrauchen_nichts prüft genau das.

UseForwardedHeaders steht ganz vorne, damit alles Spätere (Logs, Rate Limiting) schon die echte Adresse sieht.

6 Das Proxy-Problem

Von außen erreicht man Home Control nur über `tailscale serve`. Tailscale nimmt die HTTPS-Verbindung an und leitet sie an `http://127.0.0.1:5200` weiter. Für das Backend kommt deshalb **jede** Anfrage von 127.0.0.1, egal ob vom iPhone, iPad oder dem OVH-Server.

```
iPhone 100.a.b.c ──HTTPS──▶ tailscale serve ──HTTP──▶ Backend (sieht: 127.0.0.1)
                             setzt X-Forwarded-For: 100.a.b.c
```

Pro IP zu zählen, hieße dann: alle Geräte in einem Topf. Ein Gerät mit vielen Fehlversuchen würde alle anderen aussperren. Die Lösung ist der Header **X-Forwarded-For**, in den der Proxy die echte Adresse schreibt.

Vorher geprüft, nicht geraten

Header	Was tailscale serve macht (gemessen am 02.11.2026)
X-Forwarded-For	immer gesetzt auf die Tailscale-IP des Geräts; ein gefälschter Wert vom Client wird ersetzt
Tailscale-User-Login	Login des Benutzers; gefälschter Wert wird ersetzt; fehlt bei getaggten Geräten
X-Forwarded-Proto/-Host	https bzw. der Tailnet-Name (hier nicht gebraucht)

Nie blind vertrauen

Den Header kann jeder Client selbst mitschicken. Ihm zu glauben, ist nur sicher, wenn die Verbindung wirklich vom bekannten Proxy kommt:

```
builder.Services.Configure<ForwardedHeadersOptions>(options =>
{
    options.ForwardedHeaders = ForwardedHeaders.XForwardedFor;  // nur die Adresse
    options.ForwardLimit = 1;                                   // genau ein Proxy
    options.KnownIPNetworks.Clear();
    options.KnownProxies.Clear();
    options.KnownProxies.Add(IPAddress.Loopback);               // 127.0.0.1
    options.KnownProxies.Add(IPAddress.IPv6Loopback);           // ::1
});
```

Kommt eine Verbindung von einer anderen Adresse, ignoriert ASP.NET Core den Header. Gegenprobe beim Bau: Mit „allen Adressen vertrauen“ wurde der Test X_Forwarded_For_von_fremdem_Rechner_wird_ignoriert rot.

Und warum ist Tuya trotzdem global?

Bei der Regel `tuya` wird nicht ein Gerät geschützt, sondern das **Tuya-Kontingent**, und das gibt es nur einmal. Pro Gerät hieße: Anzahl Geräte × Grenze. Ein gemeinsamer Zähler schützt das Kontingent unabhängig davon, woher die Last kommt.

7 Was Rate Limiting nicht löst

Die Kontosperre (5 Fehlversuche) lässt sich weiterhin auslösen, denn 5 Versuche liegen unter der Grenze von 10 pro Minute. Rate Limiting bremst Passwort-Raten über viele Konten stark, ein gezieltes Sperren eines einzelnen Kontos verhindert es aber nicht. Eine Grenze unter 5 wäre für echte Tippfehler zu streng.

8 Logs

```
warn: Anmeldung fehlgeschlagen (falsche Zugangsdaten), Benutzer 3f2a…, von 100.a.b.c
warn: Anmeldung fehlgeschlagen (falsche Zugangsdaten), Benutzer unbekannt, von 100.a.b.c
warn: Rate Limit login überschritten: POST /api/auth/login von 100.a.b.c
```

- **Ins Log:** was passiert ist, welcher Benutzer (Id) und woher.
- **Nie ins Log:** Passwort, Token, die eingegebene E-Mail (dort landet manchmal versehentlich das Passwort).
- Die Benutzer-Id steht nur im Log, nicht in der HTTP-Antwort, sonst wüsste man, welche E-Mails es gibt.

9 Tests

Test	prüft
Login_nach_N_Versuchen_429_mit_Retry_After	3 × 401, dann 429 mit Retry-After und deutschem Text
Login_Register_und_Refresh_teilen_sich_einen_Zaehler	ein Zähler für alle drei Endpunkte
Anderes_Geraet_hat_ein_eigenes_Kontingent	Gerät B (anderes X-Forwarded-For) ist nicht betroffen

Test	prüft
X_Forwarded_For_von_fremdem_Rechner_wird_ignoriert	erfundene Header helfen einem Angreifer nicht
Tuya_hat_einen_gemeinsamen_Zaehler_fuer_alle_Geraete	Gerät A verbraucht, Gerät B bekommt 429
Tuya_Anfragen_ohne_Anmeldung_verbrauchen_nichts	Reihenfolge: Autorisierung vor Rate Limiter
Status_hat_kein_Rate_Limit	/api/status antwortet immer mit 200
Ungueltige_Grenze_verhindert_den_Start	PermitLimit 0 → App startet nicht
LoginLoggingTests (3 Tests)	Warnung im Log, ohne Passwort und ohne E-Mail

Die normale Testfabrik setzt großzügige Grenzen (100 000), sonst liefen die vielen Logins der übrigen Tests in 429. Weil die Grenzen erst bei der Anfrage gelesen werden, reicht dafür `ConfigureAppConfiguration`. Werte, die Program.cs schon beim Aufbau liest (z. B. Jwt:SigningKey), müssen dagegen per `UseSetting` gesetzt werden. Der TestServer hat keine echte Client-Adresse; `FakeRemoteIpStartupFilter` setzt sie wie hinter tailscale serve auf 127.0.0.1.

10 Kurzantworten für Rückfragen

Frage	Antwort
Was ist Rate Limiting?	Eine Obergrenze für Anfragen pro Zeit. Darüber gibt es 429.
Warum Fixed Window?	Am einfachsten zu erklären, Retry-After eindeutig. Der Randeffekt ist hier unkritisch.
Unterschied Token Bucket?	Marken werden laufend nachgefüllt: erlaubt kurze Spitzen, danach gleichmäßig.
Warum 429 und nicht 503?	429 heißt „du schickst zu viel“, 503 „der Server kann gerade nicht“.
Wozu Retry-After?	Der Client weiß, wann er es wieder versuchen darf, statt blind zu wiederholen.
Warum pro IP beim Login?	Damit ein Gerät mit Fehlversuchen die anderen nicht aussperrt.
Woher kennt das Backend die IP hinter dem Proxy?	X-Forwarded-For von tailscale serve, nur akzeptiert von 127.0.0.1/::1.
Warum Tuya global?	Geschützt wird das gemeinsame Tuya-Kontingent, nicht ein einzelnes Gerät.
Warum steht der Limiter nach der Autorisierung?	Fremde ohne Anmeldung sollen den gemeinsamen Zähler nicht leeren können.

11 Merksätze

- Rate Limiting = Obergrenze pro Zeit, darüber 429 Too Many Requests mit Retry-After.
- Partition = Schlüssel für den Zähler: pro Gerät (IP) oder für alle gemeinsam.
- Hinter einem Proxy sieht das Backend nur den Proxy. X-Forwarded-For nur vom bekannten Proxy glauben.
- Reihenfolge: ForwardedHeaders zuerst, Rate Limiter nach der Autorisierung.
- Ins Log: was, wer, woher. Nie Passwort oder Token.